ĐỀ ÁN DỰ THI SICT INNOVATION CHALLENGE 2026

TRACK: TECH YOUR SOLUTIONS (GIẢI PHÁP KỸ THUẬT SỐ)

> **Tên dự án:** Slug — Nền Tảng Chia Sẻ Workspace An Toàn & Quản Lý Cấu Hình Nhạy Cảm Trong Bộ Nhớ
> **Lĩnh vực:** Phần mềm (Quy trình làm việc & Quản lý dữ liệu) & Mạng (Chia sẻ cục bộ P2P & Bảo vệ biên dữ liệu)
> **Đơn vị dự thi:** Nhóm sinh viên Trường Công nghệ Thông tin và Truyền thông — Đại học Công nghiệp Hà Nội
> **Kho mã nguồn chính thức:** https://github.com/dotlinux26/slug (Apache License 2.0)

TÓM TẮT DỰ ÁN (EXECUTIVE SUMMARY)

Trong kỷ nguyên phát triển phần mềm hiện đại, sinh viên và kỹ sư phần mềm thường xuyên sử dụng các trợ lý lập trình trí tuệ nhân tạo (AI Coding Assistants như GitHub Copilot, Cursor, Claude Code) và làm việc nhóm trong mạng nội bộ (phòng lab trường học, văn phòng công ty, ký túc xá). Quy trình này làm phát sinh các khoảng trống bảo mật và bất tiện thực tế:

1. **Lộ lọt khóa bí mật khi lưu trữ và công khai:** Các tệp `.env`, khóa API đám mây (OpenAI, Gemini, AWS), token thanh toán và chuỗi kết nối cơ sở dữ liệu được lưu dưới dạng văn bản rõ (plaintext) trong thư mục dự án. Chúng dễ dàng bị các công cụ lập chỉ mục của AI đọc tự động, hoặc bị vô tình đẩy lên các nền tảng công khai dẫn đến việc bị bot tự động quét và thu thập.
2. **Ô nhiễm dữ liệu và thiếu kiểm soát khi chia sẻ qua mạng:** Việc chia sẻ nhanh mã nguồn dở dang cho bạn bè thường được thực hiện qua các công cụ truyền thống như `python -m http.server` hoặc nén file `.zip`. Cách làm này truyền tải cả hàng trăm megabyte dữ liệu rác (`node_modules/`, `build/`), không có mã hóa trên mạng Wi-Fi công cộng, và không có cơ chế ngăn chặn việc gửi nhầm các file cấu hình nhạy cảm.
3. **Lãng phí băng thông WAN và thiếu công cụ thu bài học sinh gọn nhẹ:** Khi chuyển file trong cùng mạng LAN văn phòng/trường học, người dùng vẫn phải tải qua Google Drive/Zalo (tốn băng thông Internet, phụ thuộc bên thứ 3). Trong khi đó, giảng viên cần thu bài 40–50 sinh viên trong phòng lab thiếu một giải pháp một chiều (Write-Only) vừa nhẹ, vừa tránh sinh viên xem bài của nhau.

Giải pháp Slug: Slug là một công cụ độc lập, tinh gọn (~8.4MB), cung cấp **Lớp kiểm soát biên thống nhất (Unified Workspace Boundary Controller)** cho môi trường học tập và phát triển:

- **Kiểm soát bí mật với Slug Guard & Module Cố Vấn AI:**
  - **Slug Guard (Cốt lõi xác định, hoạt động offline):** Hệ thống phòng thủ tự động bằng bộ lọc Heuristic/Regex, phân tích độ hỗn loạn toán học Shannon Entropy (phát hiện chuỗi ngẫu nhiên $H(X) \geq 4.0$) và VFS Jail chặn tệp nghi vấn ngay tại tầng đọc filesystem.
  - **Slug AI (Cố vấn mở rộng tùy chọn):** Hỗ trợ kết nối LLM (Local Ollama / Cloud API) đóng vai trò cố vấn tham khảo (advisory) khi người dùng muốn phân tích thêm ngữ cảnh, không thay thế các quy tắc lọc cứng.
- **Bảo vệ biến môi trường với EnvShield:** Cất file `.env` vào Kho trung tâm của người dùng (`~/.slug/env_vault/`), mã hóa repo thành `.env.pqc` , nạp biến môi trường trực tiếp từ bộ nhớ RAM cho tiến trình con khi chạy test (`slug env run`), loại bỏ văn bản rõ trên ổ đĩa.
- **Chia sẻ P2P có kiểm soát & Thu bài một chạm:** Tự động lọc bỏ rác build theo `.gitignore` , kiểm soát các file nhạy cảm ngay tại tầng đọc của hệ thống tệp ảo (VFS Jail), xác thực một chạm bằng mã OTP 8 chữ số tự hủy, phân quyền Write-Only (`[w]`) thu bài phòng máy, và mã hóa kênh truyền bằng chuẩn mật mã hậu lượng tử (NIST FIPS 203 ML-KEM-768).

MỤC I. ĐẶT VẤN ĐỀ THỰC TẾ VÀ TÍNH CẤP THIẾT

1.1. Các kịch bản đời thường trong học tập, phòng lab và văn phòng

Mỗi ngày, sinh viên công nghệ thông tin và lập trình viên đối mặt với các tình huống:

- **Chuyển tệp lớn qua mạng LAN:** Hai máy tính trong cùng phòng lab trường học hoặc văn phòng công ty cần chuyển file đồ án, tài liệu thiết kế dung lượng vài GB. Thay vì tận dụng mạng LAN gigabit (~125 MB/s), người dùng thường upload lên Google Drive hoặc gửi qua Zalo, tiêu hao băng thông Internet bên ngoài và phụ thuộc vào dịch vụ đám mây.
- **Bài toán thu bài tập sinh viên phòng máy:** Giảng viên cần thu bài thi/bài tập của 40–50 sinh viên nhanh gọn. Dùng Google Form/Drive thì tốn băng thông Internet; dùng `python -m http.server` thì sinh viên nhìn thấy bài của nhau và tải về chép; dùng FTP/SMB thì cấu hình tài khoản cồng kềnh, cần quyền admin hệ thống.
- **Lập trình với trợ lý AI và làm việc nhóm:** Sinh viên dùng Copilot, Cursor để viết code thử nghiệm gọi API (OpenAI, Gemini), đặt API key vào `.env` . Khi làm việc nhóm, việc nén cả thư mục đang code dở để gửi cho bạn bè gỡ lỗi rất dễ kéo theo cả API key thật hoặc hàng trăm MB thư mục rác `node_modules/` .
- **Yêu cầu public repository khi nộp bài:** Sinh viên nộp đồ án môn học hoặc tham gia hackathon/cuộc thi công nghệ bắt buộc phải công khai repo trên GitHub. Nếu không cẩn thận, khóa bí mật có thể bị lộ lên kho mã nguồn công khai.

1.2. Phân tích lỗ hổng của các giải pháp truyền thống

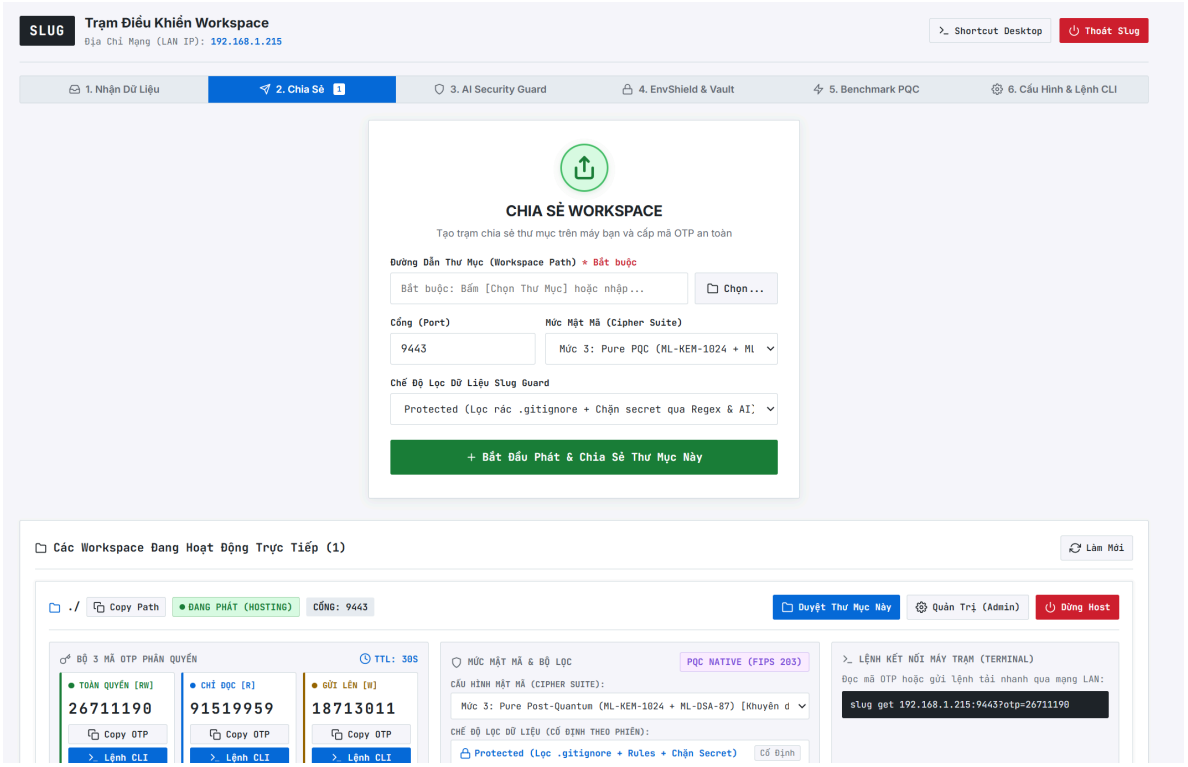
Phương thức	Lỗ hổng & Bất tiện thực tế
`python -m http.server`	• **Plaintext HTTP:** Ai cùng mạng Wi-Fi cũng bắt được gói tin mã nguồn. • **Không phân quyền:** Bất kỳ ai biết IP đều tải được mọi file. • **Không hỗ trợ nộp bài an toàn:** Chỉ cho download. Nếu dùng script upload thì ai cũng xem và tải được bài người khác vừa nộp. • **Không kiểm duyệt:** Phơi bày toàn bộ `.env` và file nhạy cảm nếu mở từ thư mục cha.
SFTP / FTP / SMB	• **Cấu hình cồng kềnh:** Đòi hỏi quyền quản trị viên (root/admin), tạo tài khoản user, mở port firewall phức tạp. • **Không cơ động (Ephemeral):** Không phù hợp để chia sẻ nhanh vài phút giữa hai máy cá nhân. • **Thiếu kiểm soát nội dung:** Không hiểu `.gitignore`, không có cơ chế rà soát secret chủ động.
Google Drive / Zalo	• **Tiêu hao băng thông Internet:** Dữ liệu nội bộ phải đi vòng qua máy chủ bên ngoài. • **Rủi ro lộ bí mật:** Khóa bí mật bị đẩy lên máy chủ bên thứ ba. • **Giới hạn dung lượng và tốc độ.**

MỤC II. TÍNH KHẢ THI VÀ HIỆN THỰC HÓA SẢN PHẨM

Slug được **hiện thực hóa hoàn chỉnh bằng mã nguồn C++20**, có sẵn bản phát hành độc lập chạy trên cả Windows và Linux.

2.1. Kiến trúc tối giản, sẵn sàng sử dụng ngay

- **Zero-Dependency:** Đóng gói thành đúng **1 file thực thi duy nhất (`slug`) với dung lượng chỉ ~8.4MB**.
- **Không cần cài đặt môi trường:** Không đòi hỏi cài Node.js runtime, Python, Java, hay cơ sở dữ liệu bên ngoài. Tải về là chạy ngay.
- **Tích hợp Web Explorer:** Ngoài giao diện dòng lệnh (CLI), Slug tích hợp sẵn một Engine Viewing nội bộ hiển thị trên trình duyệt (`127.0.0.1:8080/drop`) với phong cách Techno B2B trực quan, hỗ trợ xem cây thư mục và tải file một chạm.



Hình 1: Giao diện Web Viewing Engine & Dashboard chia sẻ Workspace trực quan của Slug.

2.2. Quy trình sử dụng 3 bước tinh gọn (The 3-Step Workflow)

Sản phẩm được thiết kế sao cho người dùng có thể làm quen nhanh chóng:

- **Bước 1 — Quét & Bảo vệ bí mật:**

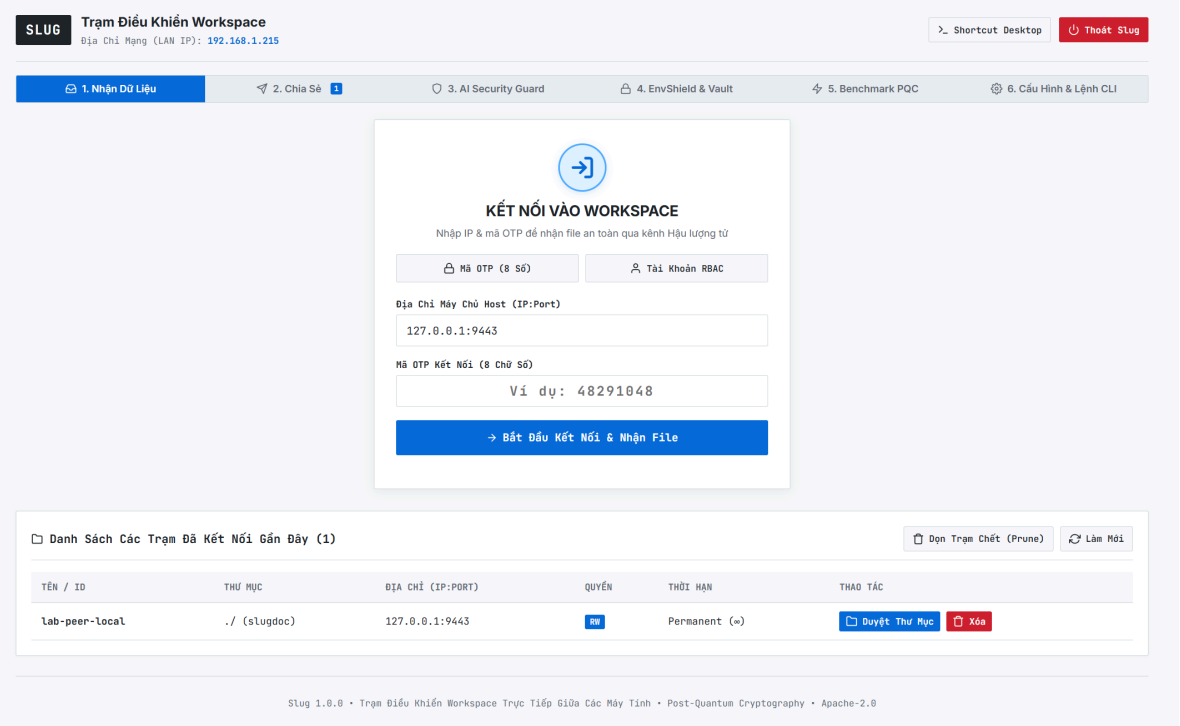
```bash
slug audit scan .                # Quét phát hiện secret qua Regex & Shannon Entropy
slug env init                    # Mã hóa .env thành .env.pqc, xóa plaintext trên đĩa
```

- **Bước 2 — Khởi tạo trạm phát P2P:**

```bash
slug add . -p 9443               # Mở trạm chia sẻ có lọc rác build, cấp mã OTP 8 số
```

- **Bước 3 — Máy nhận kết nối và tải về:**

```bash
slug pull -i <IP> -p 9443 -o <OTP> --to ./du_an
# Hoặc mở giao diện Web trên trình duyệt: gõ lệnh 'slug' -> http://127.0.0.1:8080/
```



Hình 2: Trạng thái kết nối và bắt tay trao đổi khóa an toàn giữa Client và Host.

MỤC III. TÍNH SÁNG TẠO VÀ CÁC ĐIỂM MỚI ĐỘT PHÁ

3.1. Cơ chế niêm phong và khôi phục biến môi trường (PQC EnvShield)

- Tự động di chuyển bản sao `.env` ra ngoài workspace vào thư mục an toàn của người dùng: `~/.slug/env_vault/<workspace_hash>/` .
- Thư mục dự án chỉ còn lại file `.env.pqc` (được mã hóa bằng ML-KEM-768 và XChaCha20-Poly1305). File bản rõ trên đĩa bị xóa.
- Khi cần sửa đổi hoặc tương tác với các công cụ bắt buộc đọc file đĩa, sinh viên chỉ cần gõ `slug env restore` để khôi phục nhanh chóng.

3.2. Cấp biến môi trường trực tiếp từ RAM (Process RAM Injection)

Lệnh `slug env run -- <lệnh>` đọc và giải mã `.env.pqc` trong bộ nhớ RAM của Slug, sau đó nạp trực tiếp vào khối môi trường của tiến trình con thông qua API hệ điều hành (`CreateProcessW` trên Windows, `execve` trên Linux).

- Tiến trình con (ví dụ `pytest`) nhận đầy đủ biến môi trường để chạy test bình thường.
- Không sinh ra tệp bản rõ tạm thời trên ổ đĩa.

3.3. Cổng kiểm soát đọc VFS Jail (Closed-Loop Read Path Gating)

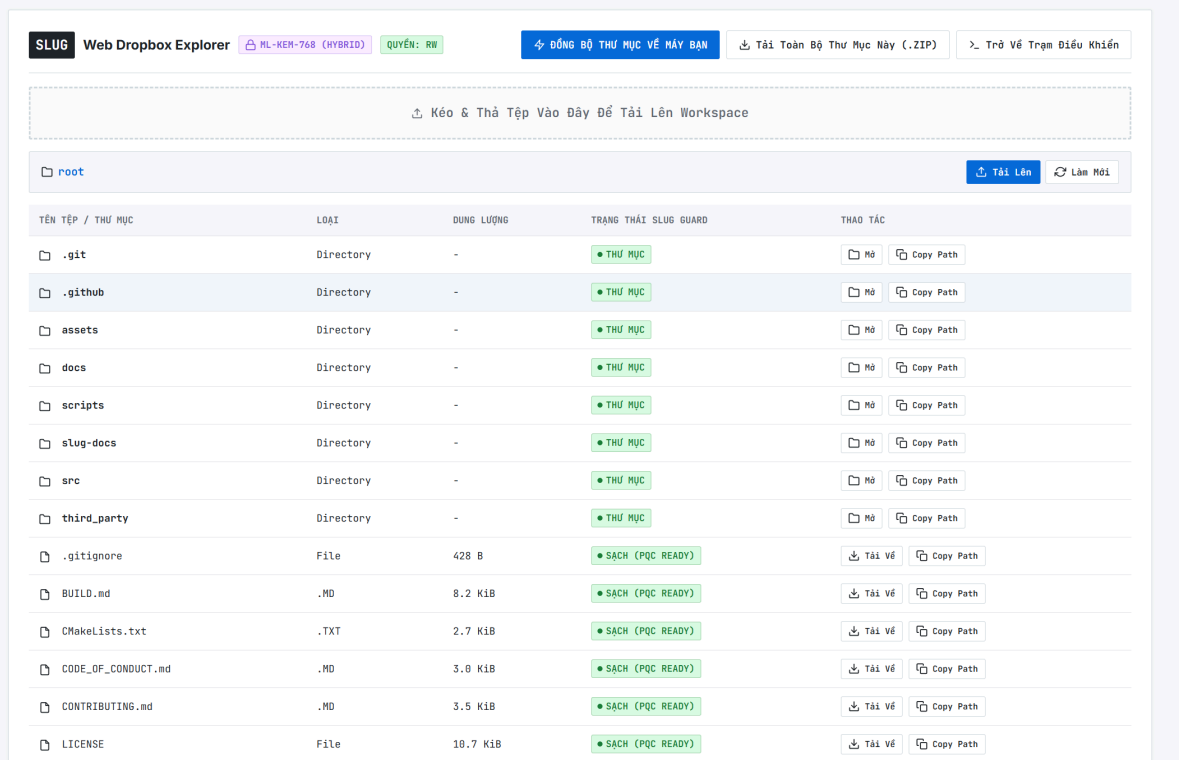
Trong các công cụ truyền file thông thường, nếu người dùng quên quét secret, công cụ vẫn gửi file đi. Ở Slug, hệ thống tệp ảo (`SlugVFS`) kiểm soát chặt chẽ hàm đọc dữ liệu

`vfs_.read_file()` :

- Một file bị đánh dấu cách ly (`QUARANTINED` / `RULE_BLOCKED`) sẽ bị từ chối ngay tại ranh giới filesystem.
- Kể cả khi client cố tình gửi request tải trực tiếp một chunk của file đó mà bỏ qua manifest, máy chủ Slug vẫn chặn tại chỗ.

3.4. Thu bài tập một chiều Write-Only (`[w]` Blind Drop)

- Giáo viên chỉ cần chạy `slug add ./bai_nop -p 9443` và cấp tài khoản chung quyền `[w]` (hoặc cấp mã OTP `[w]` cho từng máy) cho sinh viên.
- Sinh viên chỉ có quyền tải bài lên qua Web Dropbox hoặc lệnh `slug push` .
- Tầng `SlugVFS Jail` chặn quyền đọc (`read_file`) và liệt kê (`list_dir`) đối với role này. Sinh viên không thể xem hoặc tải bài của nhau.



Hình 3: Giao diện hòm thư nộp bài một chiều (Blind Drop) hỗ trợ thu bài phòng máy, chống xem bài của nhau.

3.5. Ba chế độ chia sẻ linh hoạt (Configurable Filter Modes)

- `protected` (Mặc định): Bật lá chắn lọc `.gitignore` , chặn secret và cách ly dữ liệu nhạy cảm.
- `include-ignored` (`--include-ignored`): Chia sẻ cả file build/artifact bị gitignore, nhưng vẫn giữ rào chắn chặn secret.
- `none` (`--no-filter`): Chia sẻ thô cho máy tin cậy, nhưng các cơ chế an toàn cơ bản luôn được duy trì: không cho phép path traversal vượt ra khỏi thư mục gốc (`resolve_safe_path`), không để lộ thư mục cấu hình nội bộ `.slug/` .

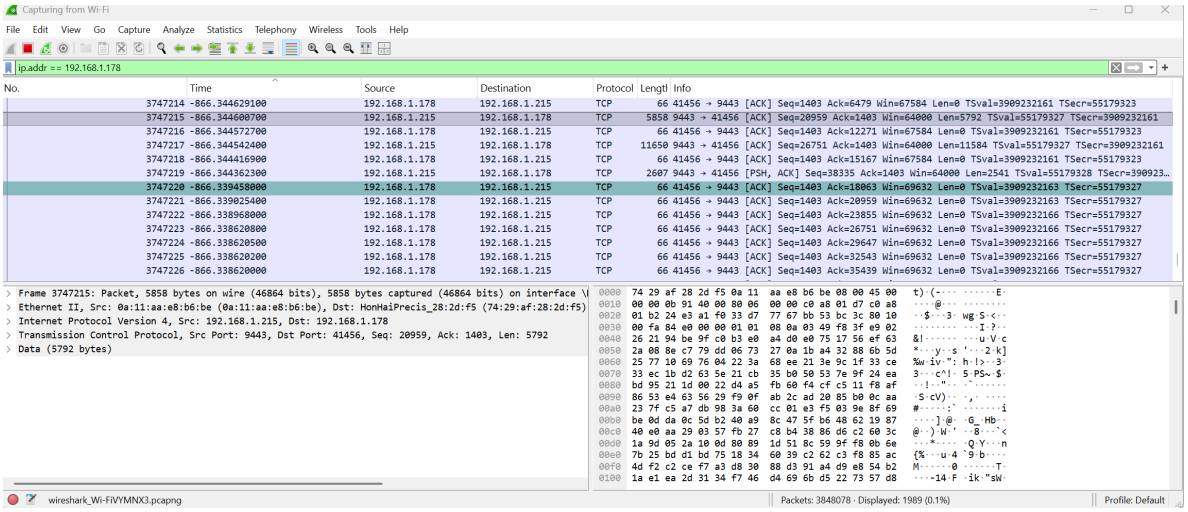
MỤC IV. PHÂN ĐỊNH BẢO MẬT: SLUG GUARD VÀ SLUG AI

Slug phân định ranh giới kỹ thuật rõ ràng giữa hai tầng:

4.1. Slug Guard: Tầng phòng thủ cốt lõi, xác định và hoạt động offline

Cơ chế bảo vệ dữ liệu của Slug trong việc ngăn chặn lộ lọt bí mật do Slug Guard đảm nhiệm:

1. **Bộ lọc Heuristic & Mẫu định danh (Deterministic Regex):** Tự động nhận diện các mẫu credential phổ biến: AWS Access Key (`AKIA...`), GitHub Token (`ghp_...`), Private Key Headers (`BEGIN PRIVATE KEY`), Chuỗi kết nối Database có mật khẩu, JWT, Stripe keys, v.v.
2. **Phân tích độ hỗn loạn toán học (Shannon Entropy Analyzer):**
   - Đo lường độ ngẫu nhiên thông tin: $H(X) = -\sum P(xi) \log 2\, P(xi)$.
   - Bất kỳ chuỗi ký tự nào có độ dài ≥ 20 ký tự và entropy $H(X) \geq 4.0$ bits/byte sẽ được gắn cờ nghi vấn là token bí mật/khóa mật mã, kể cả khi chuỗi đó không theo định dạng regex định trước.
3. **Phân loại tệp & Miễn trừ tệp mẫu:** Tự động chặn các tệp cấu hình bí mật (`.env` , `.env.local` , `.pem` , `.key`), đồng thời hỗ trợ miễn trừ các tệp mẫu tài liệu (`.env.example` , `.env.sample` , `.env.template`) để hạn chế báo động giả (False Positive).
4. **VFS Jail & Manual Guard:** Nếu file bị đánh dấu `QUARANTINED` hoặc `RULE_BLOCKED` , hệ thống tệp ảo khóa hàm đọc `vfs_.read_file()` , ngăn dữ liệu bị gửi qua mạng. Chủ workspace có quyền chủ động Whitelist (tiền tố `!`) hoặc chặn thủ công từng file.



Hình 4: Kiểm thử thực nghiệm với Wireshark trên mạng LAN: Dữ liệu truyền tải được mã hóa nhị phân PQC, ngăn chặn hoàn toàn việc nghe lén.

4.2. Slug AI: Tầng cố vấn ngữ nghĩa tùy chọn (Optional Contextual Advisory)

Slug AI là một module tiện ích mở rộng tùy chọn, kích hoạt khi người dùng truyền cờ `--ai` và có cấu hình kết nối mô hình (Local Ollama hoặc Cloud API):

- **Bản chất kỹ thuật:** Slug trích xuất trích đoạn mã và gửi prompt kỹ thuật tới LLM, nhận phản hồi dạng JSON có cấu trúc (`has_secret` , `reason` , `line`).
- **Vai trò cố vấn độc lập (Advisory Only):** AI đóng vai trò ý kiến tham khảo hỗ trợ người dùng xem xét ngữ cảnh (ví dụ: chuỗi trong thư mục test hay chuỗi thực thi).
- **Ranh giới an toàn:** Slug không phó thác an toàn hệ thống cho AI. AI không có quyền quyết định đóng/mở kênh mạng, và AI không thể vượt qua các quy tắc cấm cứng của Slug Guard hay chính sách của chủ máy trong `.slug/rules.conf` .

4.3. Bộ nhớ đệm kiểm toán băm BLAKE3 (Stateful Audit Index)

- Slug duy trì cơ sở dữ liệu trạng thái (`.slug/index.db`) gắn với mã băm nội dung BLAKE3 của từng file.
- **Tối ưu thời gian:** File chưa bị sửa đổi sẽ lấy kết quả kiểm toán từ cache trong thời gian ngắn.
- **Xác thực toàn vẹn (Anti-TOCTOU):** Nếu người dùng sửa nội dung file, mã băm BLAKE3 thay đổi → trạng thái chuyển sang `STALE` , buộc phải kiểm tra lại trước khi cho phép chia sẻ qua mạng.

4.4. Hướng dẫn công cụ cho AI Agent (Agent Directive)

- Khi chạy `slug env init` , Slug tạo file chỉ thị `.slug/ai-rule.md` (và lệnh `slug env prompt`).
- Chỉ thị này định hướng các AI Coding Agent bên ngoài (như Cursor, Copilot, Claude Code) thực thi lệnh qua `slug env run -- <lệnh>` , tránh việc cố đọc file cấu hình bản rõ từ đĩa cứng.

MỤC V. GIẢI THÍCH KỸ THUẬT VÀ CÂU HỎI THƯỜNG GẶP

1. Cơ chế hoạt động của `slug env run` với các loại ứng dụng

- **Chi tiết kỹ thuật:** Đa số các ứng dụng và framework hiện đại (Node.js, Python, Go, Rust, C++) đọc từ biến môi trường trên RAM (`process.env` , `os.environ`), do đó hoạt động bình thường qua `slug env run` .
- **Phạm vi:** Đối với các công cụ hoặc thư viện có đoạn mã cố định bắt buộc phải mở file `.env` trên ổ đĩa (`fopen("./.env")`), ứng dụng sẽ báo thiếu file. Trong trường hợp này, người dùng có thể dùng `slug env restore` để tạm thời khôi phục file `.env` từ Central Vault khi cần thiết, và dùng `slug env init` để niêm phong lại sau đó.

2. So sánh với các công cụ quét secret độc lập (như Gitleaks)

- **Chi tiết kỹ thuật:** Gitleaks là công cụ kiểm tra độc lập, kết quả quét tách rời khỏi công cụ truyền file. Nếu lập trình viên quên chạy kiểm tra mà trực tiếp mở `http.server` hoặc gửi

zip, file cấu hình vẫn có thể bị gửi đi.
- **Điểm khác biệt:** Slug liên kết trực tiếp kết quả kiểm duyệt với hệ thống tệp ảo (`SlugVFS`). File chưa đạt kiểm duyệt sẽ bị VFS từ chối phục vụ ngay ở tầng đọc gói tin.

3. Vai trò của hàm băm BLAKE3 trong hệ thống

- **Chi tiết kỹ thuật:** BLAKE3 là hàm băm kiểm tra tính toàn vẹn nội dung, không xác định tính nhạy cảm của dữ liệu.
- **Giá trị thực tế:** BLAKE3 trong Slug đóng vai trò vô hiệu hóa bộ nhớ đệm (Cache Invalidation). Trạng thái kiểm duyệt chỉ có hiệu lực với đúng phiên bản nội dung đã được quét; nếu nội dung bị sửa đổi, trạng thái chuyển thành `STALE` và cần được đánh giá lại.

4. Sự kết hợp giữa ML-KEM-768 và mã xác thực OTP

- **Chi tiết kỹ thuật:** ML-KEM-768 (NIST FIPS 203) là cơ chế đóng gói khóa để thiết lập kênh truyền mã hóa an toàn trước máy tính lượng tử. KEM thiết lập kênh bí mật nhưng không định danh người nhận cụ thể.
- **Cơ chế phối hợp:** Slug sử dụng mã OTP 8 số (dựa trên RFC 6238 TOTP) trao đổi ngoài kênh (out-of-band) để xác thực người nhận. Khi client kết nối, mã OTP được gửi bên trong kênh PQC đã mã hóa và được máy chủ tiêu thụ theo cơ chế dùng một lần: sau khi dùng sẽ hết hiệu lực, tránh tấn công phát lại (Replay Attack).

5. Phân định vai trò giữa Slug Guard và module AI

- **Chi tiết kỹ thuật:**
  - **Slug Guard là lớp bảo vệ xác định:** Hoạt động offline, dựa trên phân tích Shannon Entropy, quy tắc regex và khóa VFS Jail.
  - **Slug AI là cố vấn tùy chọn:** Hỗ trợ người dùng tham khảo thêm thông tin ngữ nghĩa khi có kết nối LLM. Hệ thống Slug Guard vẫn duy trì kiểm soát bảo vệ cho workspace độc lập với module AI.

MỤC VI. KỊCH BẢN THỰC NGHIỆM VÀ HƯỚNG DẪN DEMO 3 PHÚT

Quy trình trình diễn thực nghiệm:

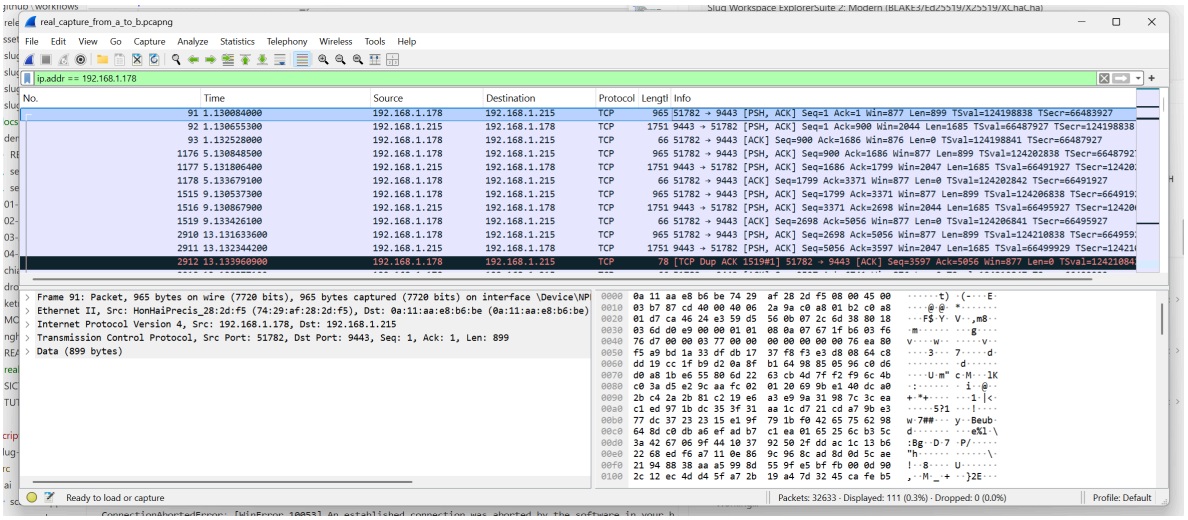
```text
+-------------------------------------------------------------------------------
+
| KỊCH BẢN TRÌNH DIỄN THỰC NGHIỆM (DEMO WALKTHROUGH)
|
+-------------------------------------------------------------------------------
```

```
+

[PHÚT 1: QUẢN LÝ BIẾN MÔI TRƯỜNG VỚI ENVSHIELD]
1. Mở thư mục dự án mẫu có chứa file `.env` với các khóa cấu hình.
2. Gõ lệnh: slug env init
   -> File `.env` bản rõ được xóa khỏi thư mục dự án, thay bằng `.env.pqc`.
   -> File gốc được lưu tại ~/.slug/env_vault/.
3. Chạy test ứng dụng: slug env run -- pytest
   -> Ứng dụng chạy thành công: secret được nạp thẳng vào RAM mà không cần file đĩa.

[PHÚT 2: PHÁT HIỆN SECRET QUA SLUG GUARD & THAM VẤN NGỮ CẢNH]
1. Tạo một file mã nguồn mới `src/leak.py` chứa một API key mẫu và một chuỗi token ngẫu nhiên.
2. Gõ lệnh: slug audit scan .
   -> Slug Guard phát hiện chuỗi có Shannon Entropy >= 4.0 và khớp mẫu khóa bí mật.
   -> Trạng thái file chuyển sang QUARANTINED.
   -> Nếu có cờ --ai: Module cố vấn hỗ trợ phân tích có cấu trúc JSON để người dùng tham khảo.

[PHÚT 3: CHIA SẺ P2P CÓ KIỂM SOÁT VÀ THU BÀI]
1. Trên máy gửi, gõ: slug add .
   -> Thông báo tự động bỏ qua node_modules/.
   -> File `src/leak.py` bị cách ly, không có mặt trong danh sách chia sẻ.
   -> Xuất hiện link kèm mã OTP 8 chữ số.
2. Trên máy nhận, gõ: slug pull -i <IP> -p 9443 -o <OTP> --to ./du_an
   (hoặc mở Web Viewing Engine trên loopback bằng lệnh 'slug').
   -> Dữ liệu được kéo về máy nhận.
   -> Trên máy nhận: Không có thư mục rác node_modules/, không có .env và file bị cách ly.
3. Thử kết nối lại bằng mã OTP cũ:
   -> Máy chủ thông báo OTP đã hết hạn hoặc đã được sử dụng.
```



Hình 5: Phân tích chi tiết luồng gói tin thực tế truyền giữa máy phát (Host) và máy nhận (Client).

MỤC VII. KẾT LUẬN VÀ THÔNG TIN DỰ ÁN

Slug hướng đến giải quyết các bài toán thực tế trong môi trường học tập và phát triển phần mềm:

1. **Phù hợp với chủ đề SICT Innovation Challenge:** Giải quyết vấn đề trong cả hai lĩnh vực **Phần mềm** (quy trình làm việc, dữ liệu, biến môi trường) và **Mạng** (kết nối P2P, băng thông, bảo mật đường truyền).
2. **Khả thi và hoàn thiện:** Sản phẩm là ứng dụng C++20 hoàn chỉnh, chạy thực tế với hiệu năng cao, đóng gói 1 file nhị phân duy nhất ~8.4MB.
3. **Phân định AI rõ ràng:** Đặt nền tảng an toàn trên các quy tắc xác định của Slug Guard; định vị AI đúng vai trò cố vấn ngữ nghĩa tùy chọn.

Thông tin mã nguồn và tài liệu kỹ thuật:

- **Repository chính thức:** https://github.com/dotlinux26/slug
- **Giấy phép bản quyền:** Apache License 2.0 (Mã nguồn mở phi lợi nhuận).